AAS 25-360

ASTROROS : INTEGRATED SIMULATION FRAMEWORK FOR SPACE ROBOTIC SYSTEM

Minsik Oh[*], Heokjune You[†], and Dongwon Jung[‡]

On-orbit service missions enable sustainable space operations in the future, by providing services such as refueling, repair and space debris removal. These operations involve significant technical challenges, including autonomous rendezvous and proximity operation, precise docking, and robotic manipulation. However, the high cost and limited accessibility of space environments make it difficult to integrate and test such systems under real-world condition. Therefore, simulation-based approaches are essential as an initial step to ensure feasibility and optimize system level design. In this study, an integrated mission simulation framework named *AstroROS*, which utilizes ROS/Gazebo environment is presented. AstroROS is developed to monitor changes in satellite orbits and states caused by thrust or collisions, while ensuring the integrated and effective operation of various onboard control systems under these conditions. The *AstroROS* consists of four component modules: object manager, orbit simulator, data flow manager, and physics simulator. Object manager describes object's components and spawns object in physics simulator. If arbitrary physical events exist in physics simulator, such as actuator's force/torque or collisions, they are fed into the orbit simulator to apply the resulting orbital changes. Data flow is managed by the data flow manager, which supervises all information generated during the simulation. As a case study within the AstroROS framework, a mission operation simulation is conducted using a satellite equipped with a ClearSpace-like tentacle arm and an attitude control system.

INTRODUCTION

Recently, the On-Orbit Service(OOS) mission has received a great deal of attention from the space-development community. These missions include Active Debris Removal(ADR), refueling, proximity operation, etc. The success of such missions depends on the development and implementation of core technologies, such as Guidance, Navigation, and Control (GNC) systems for autonomous rendezvous and docking. In recent years, space robotic systems have also emerged as critical technologies, driven by the increasing complexity of missions like space debris removal. While each technology can be developed and tested individually, it is essential to validate complete system operation within a simulated mission environment. For this purpose, the air bearing simulator facility can provide a unique and effective environment for the validation of hardware and software technologies.[1,2] However, this method is constrained by high costs and difficulties in replicating the 6-DoF space environment, which limits its application.[3]

Consequently, simulation-based approaches have been widely used in recent research to handle these challenges and provide cost-effective alternatives. Among these, "*Basilisk*"[4] has gained at-

[*]Department of Smart Air Mobility Engineering, Korea Aerospace University
[†]Department of Smart Air Mobility Engineering, Korea Aerospace University
[‡]Department of Smart Air Mobility Engineering, Korea Aerospace University

tention as a flexible, modular astrodynamics simulation framework capable of modeling complex spacecraft systems with high fidelity throughout all phases of spacecraft Attitude Determination and Control System (ADCS) design and validation. While Basilisk excels at high-fidelity astrodynamics simulations, there has also been growing interest in leveraging more general physics simulators that focus on other critical tasks, such as contact dynamics and berthing, particularly using the ROS/Gazebo system; Robot Operating System that facilitates fast and modular program application, adhering to the principle "*Don't re-invent the wheel*", while Gazebo is an open-source physics simulator widely used in robotics field. Chow et al.[5] simulated docking and berthing operations using a ROS/Gazebo system incorporating lidar and depth camera with robot arm in Gazebo environment. Similarly, Ramon et al.[3] designed a simulation framework named "*OnOrbitROS*", which implements spacecraft attitude perturbations due to orbital movement and validates compute-based controllers for humanoid space robots like NASA's Valkyrie.[6] More recently, NASA introduced a ROS2/Gazebo-based space simulation framework named "*SpaceROS*", an open-source spacecraft flight software framework for space robotic systems, developed in collaboration with Blue Origin.[7] SpaceROS aims to shorten development timelines and enhance code re-usability, thereby allowing the space community to capitalize on the extensive resources and tools available within the ROS community. Following the trend of open-source developments in the space community, this paper introduces"*AstroROS*", a novel simulation framework specifically designed for OOS missions. The primary motivation behind developing AstroROS is to simulate and integrate GNC systems for operating space robotic systems, with a particular focus on the connection between orbit simulation and dynamics simulation through interconnection to account for orbital changes resulting from force and torque. The remainder of this paper is organized as follows. The first section introduces the architecture of *AstroROS* with a detailed description of each module. In second section , for testing the utility of *AstroROS*, a service spacecraft equipped with tentacle mechanism similar to ClearSpace[8] spacecraft and a target object are designed to simulate a virtual mission,including ADCS control and tentacle control for grasping the target object. Finally, the conclusion and future work are discussed in last section .

ASTROROS ARCHITECTURE

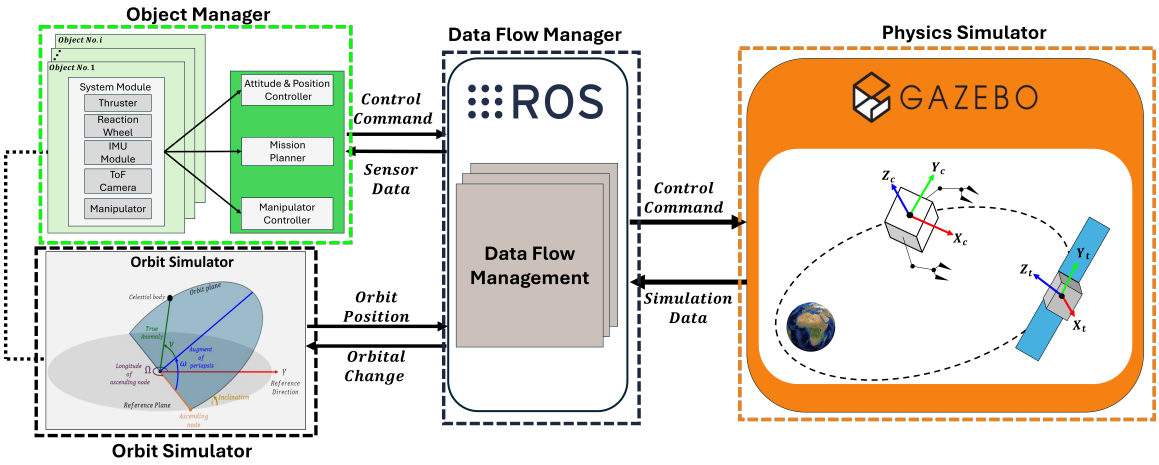


Figure 1: AstroROS Architecture

The *AstroROS*,illustrated in Fig 1, consists of four component modules: Object Manager, Orbit

simulator, Data flow manager, and Physics simulator. The object manager includes the general definitions of objects as well as the function definitions of object components. The orbit simulator calculates the orbital position of the objects using the orbit propagation engine in which the non-gravity acceleration acting on the object enables the orbital changes. The physics simulator performs a 6-DoF simulation of multiple objects, taking into account both multi-body and contact dynamics under the assumption of zero gravity. The acceleration from the physics simulator is then fed to the orbit simulator to apply orbital change of the object. In this way, force exerted on the object, such as those from thrusters, can change both the attitude of the body and its orbit simultaneously. Finally, the data flow manager acts as a data bridge to share the data seamlessly among the components module.

Object Manager

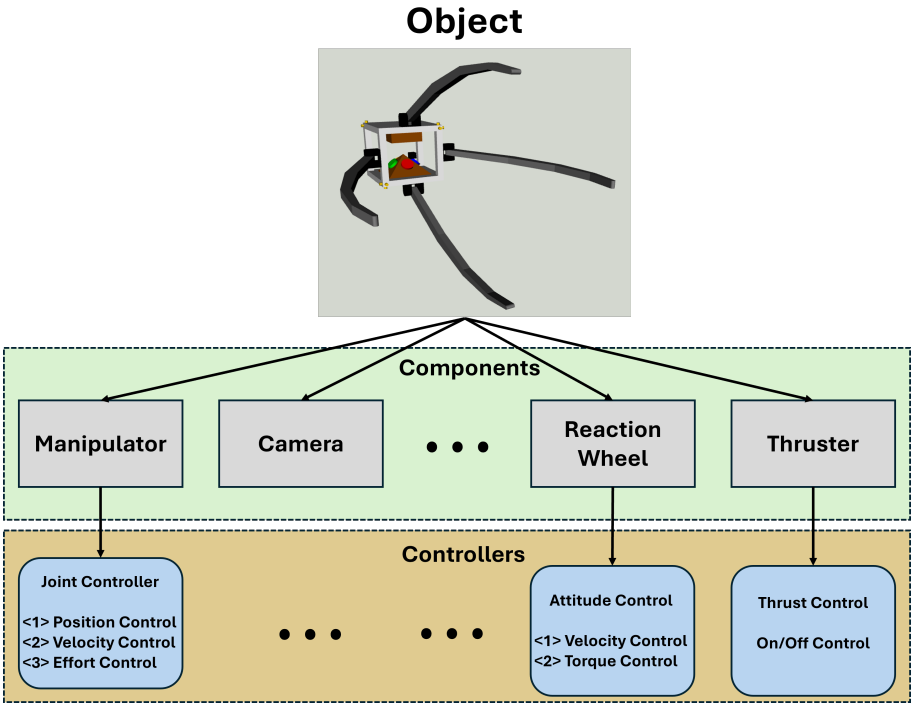


Figure 2: Structure of Object manager

The object manager is a functional module that defines the objects to be used in two simulators. Figure 2 shows a structure of the object manager. It is composed of two parts: the components and controllers. The components contain Simulation Description Format(SDF)-based xacro files, which are an extended format used for complex robot descriptions. Xacro, which stands for XML Macros, is utilized in AstroROS because it allows for the creation of more modular and reusable robot description files. For instance, common components like sensors, joints, or even entire sub-systems can be defined once and reused across multiple objects or configurations. The controllers are responsible for managing the behavior and interactions of the components within the Physics Simulator. They are also registered in the same xacro files coupling with proper components. The controllers are directly related with Physics Simulator because they are implemented by *Plugin* that will be introduced in Physics Simulator.

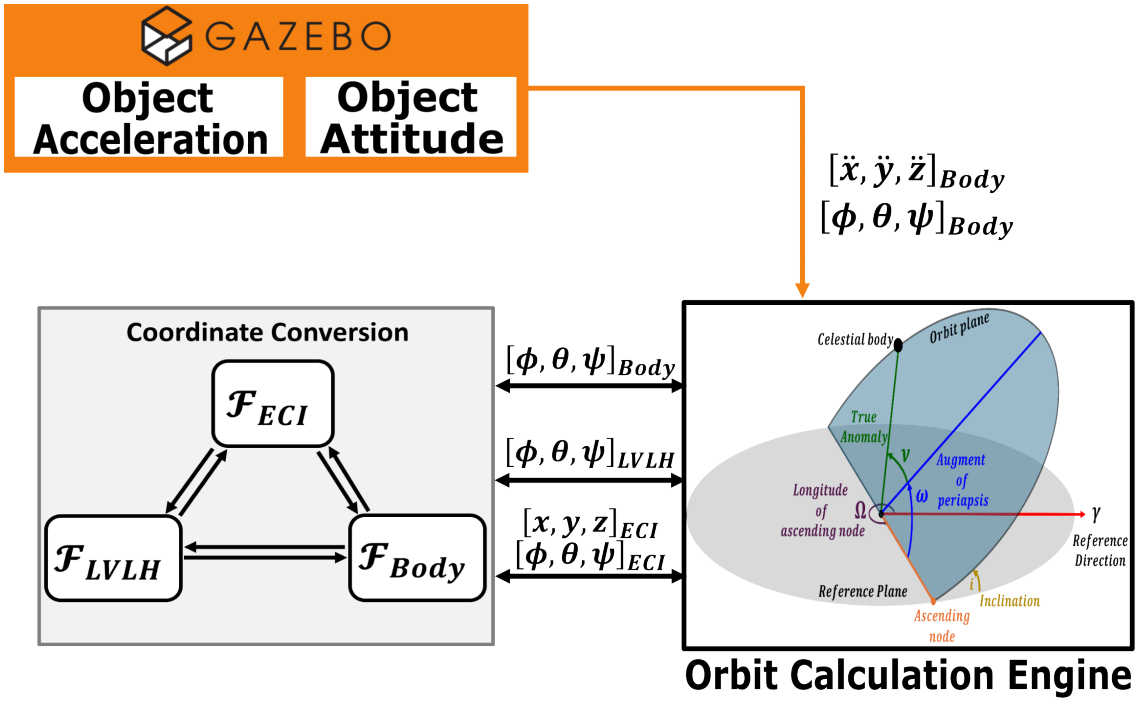


Figure 3: Flow diagram of orbit simulator

Orbit Simulator

The Orbit Simulator is an orbit propagation tool designed for analyzing the location of objects in orbit. MATLAB/Simulink is chosen to implement the Orbit Simulator due to its powerful integration of numerical analysis and extensive toolbox support that contains orbit calculation library.[9] These features make it ideal for handling the complex calculations required for accurate orbit propagation. Figure 3 shows the flow of the Orbit Simulator, which comprises two components: the Coordinate Conversion and the Orbit Calculation Engine.

The Orbit Calculation Engine is the core computational component of the Orbit Simulator. By defining an object's initial Keplerian elements and orientation, it computes the object's position over time. This engine utilizes the open-source PROPAT toolbox,[9] specifically developed for satellite orbit simulation. The toolbox offers a comprehensive suite of functions that enable various calculations based on the two-body problem, taking into account perturbations such as J2 and J4.

The Orbit Simulator is linked with the Physics Simulator, which means that physical interactions within the Physics Simulator can lead to changes in an object's position and attitude, thereby influencing the orbit calculations within the Orbit Simulator. This relationship between the two simulators allows AstroROS to perform a fully multi-body nonlinear simulation. For example, physical events such as contact, collision, or momentum exchanges caused by object body components can not only alter the object's position and orientation but also makes orbital change caused by force or torque. It could be explained that the dynamics in the Physics Simulator is computed in the body frame of the object and eventually affects the Orbit Simulator due to the relationship between the orbit frames and the body frame. In other words, these events generate force and torque, which can lead to potential changes to the object's orbit. Since these force and torque are calculated in the body

frame and cannot be directly applied to the orbit frames, the Coordinate Conversion component is necessary within the Orbit Simulator. The three frames are used; the Earth-Centered Inertia (ECI), Local-Vertical-Local-Horizontal (LVLH), and the body frame of the object. All data in the Orbit Simulator is processed through the Coordinate Conversion component to account for the relationships between these frames, enabling accurate orbit propagation calculations. The relationships and conversion processes are analyzed in detail and can be accessed in this article.[10]

Physics Simulator

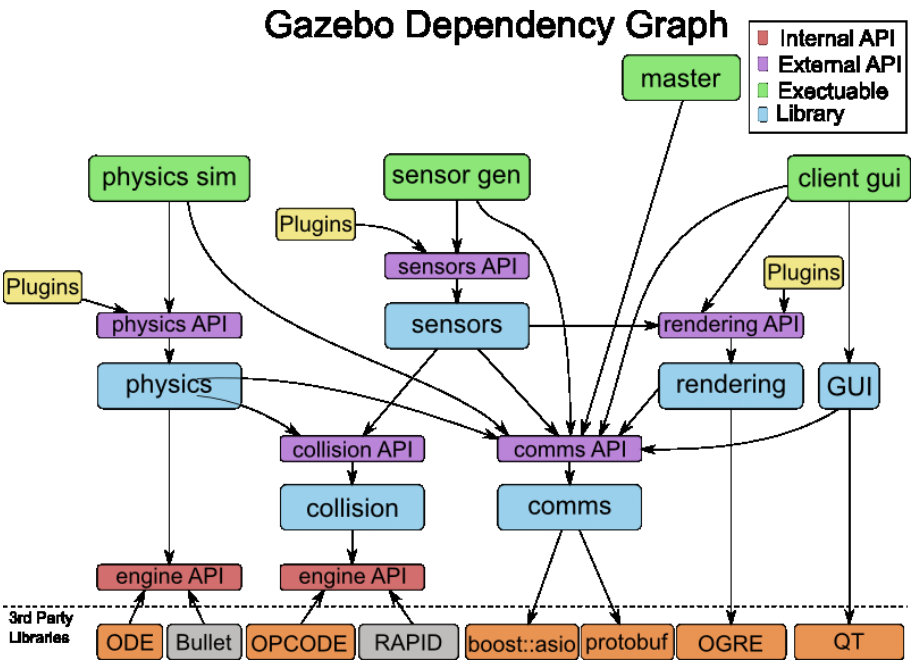


Figure 4: Architecture of Physics Simulator[11]

Physics Simulator is an essential tool for solving dynamics problems by integrating a physics engine with graphic rendering capabilities. In the field of physics simulation, several physics simulators are available like Unity and Unreal Engine etc., AstroROS employs Gazebo, a popular choice known for its seamless integration with ROS.[11] Gazebo is structured with several APIs, including the physics API, sensor API, and rendering API, as depicted in Figure 4. Especially, physics API plays a crucial role in solving dynamics problems by interfacing directly with the physics engine. Users also have the flexibility to select an appropriate physics engine from options such as ODE (Open Dynamics Engine), Bullet, and DART (Dynamic Animation and Robotics Toolkit),[12] depending on their specific requirements. In many cases, including within the AstroROS framework, ODE is the preferred choice due to its robustness and reliability in simulating rigid body dynamics.

A significant feature of Gazebo is *Plugin*, which enables customized behaviors and interactions within the simulation environment. Plugin,a custom modification feature in ROS/Gazebo,is a user-defined extension to empowers users to extend the functionality of the simulation. AstroROS takes full advantage of Plugin by using Gazebo's plugins to extend the simulator's capabilities. For instance, AstroROS employs plugins to implement sensors (*e.g.* IMU and ToF camera) and controllers (*e.g.* position/velocity controllers)[13] which are not implemented in default. This enables the development of advanced simulation features, such as custom sensors, actuators, and complex interaction

behaviors not supported by the default physics engines. This extensibility is particularly beneficial in research and development settings, where specialized simulation requirements are common.

Data Flow Manager

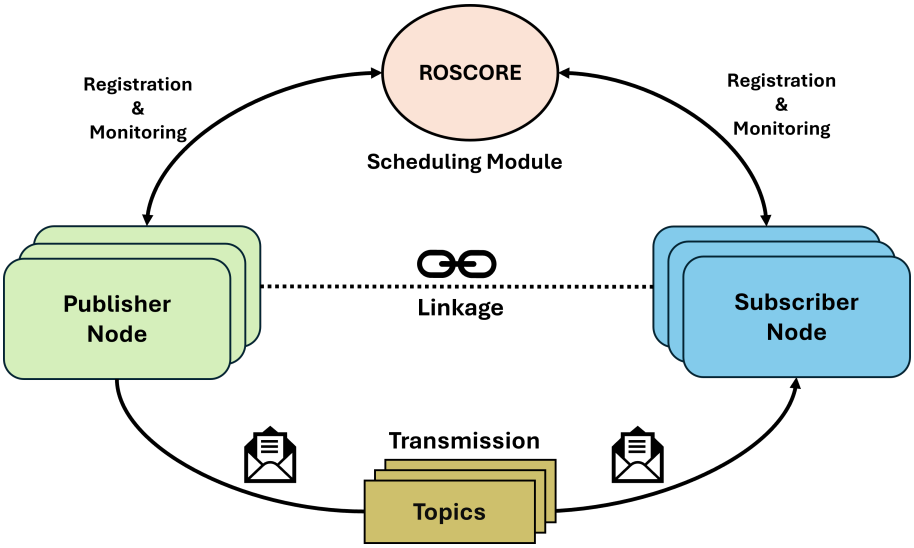
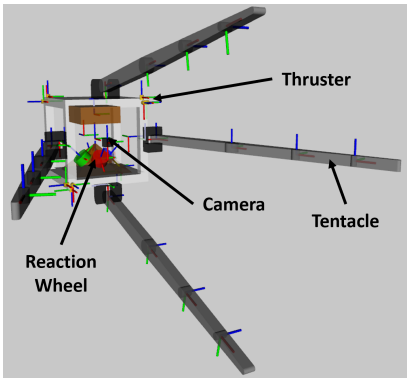


Figure 5: Mechanism of data flow manager

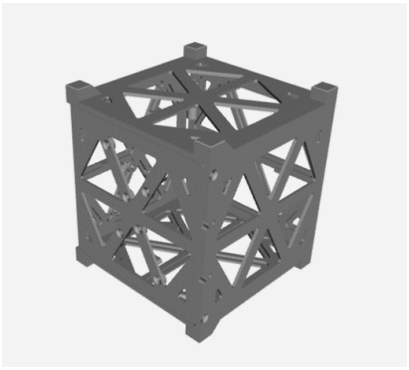
Data Flow Manager is implemented using the ROSCORE system. ROS utilizes ROSCORE with ROSTCP protocol to enable real-time data communication between its various components. Figure 5 illustrates the data flow process managed by the Data Flow Manager. Data Flow Manager provides a flexible mechanism for the exchange of data packets, known as ”topics” in ROS, between ”nodes,” which are the fundamental units responsible for generating or receiving topics. Nodes in ROS can be thought of as modular blocks, each performing specific tasks such as sensor data acquisition, data processing, or actuator control. Among the various data communication methods available in ROS, the ”Publish & Subscribe” model is the most widely used. In this model, nodes publish data on specific topics, while other nodes subscribe to these topics to receive the published data. To maintain robust data handling and ensure system reliability, ROS provides comprehensive logging and debugging tools. These tools give developers with detailed insights into the data flow and node interactions, facilitating the monitoring and troubleshooting of the system.

CASE STUDY

This section presents two virtual scenarios and simulation results to validate the functionality of AstroROS using 4 tentacle arm satellite and a target space object. The first scenario is an attitude control , which involves aligning the satellite's body frame with LVLH frame. The second scenario is an ADR mission aimed at capturing the target object. The designs of the chaser satellite and the target object are illustrated in Figure 6. The chaser is composed of thrusters, base camera, tentacle arm and reaction wheels. The thrusters provide the necessary propulsion to maneuver the satellite and perform fine adjustments to its position and orientation. The base camera is utilized for visual guidance and navigation, enabling the chaser to execute proximity operation for the target. The tentacle, inspired by the ClearSpace satellite design, serves as a flexible robotic arm capable of extending and maneuvering to capture the target object. This tentacle is equipped with sensors and



(a) Tentacle satellite



(b) Target satellite

Figure 6: Design of ClearSpace-like tentacle arm satellite and target object

actuators to ensure precise control and grip strength, crucial for securing debris of varying shapes and sizes. The reaction wheels are arranged in a pyramid configuration that provides redundancy and enhanced control torque for stabilizing the chaser's attitude. The target is configured as a simple square satellite with following assumptions; 1. The target is a rigid body. 2. The target has no tumbling motion. 3. The target can be easily observed due to the absence of light scattering effects.

Case 1 : Attitude Control

This mission is an attitude control mission aimed at aligning the chaser satellite's body frame with LVLH frame as illustrated in Figure 7. The satellite is assumed to be in a circular low Earth orbit (LEO) with an inclination of 45 degrees. The simulation is carried out over the course of one

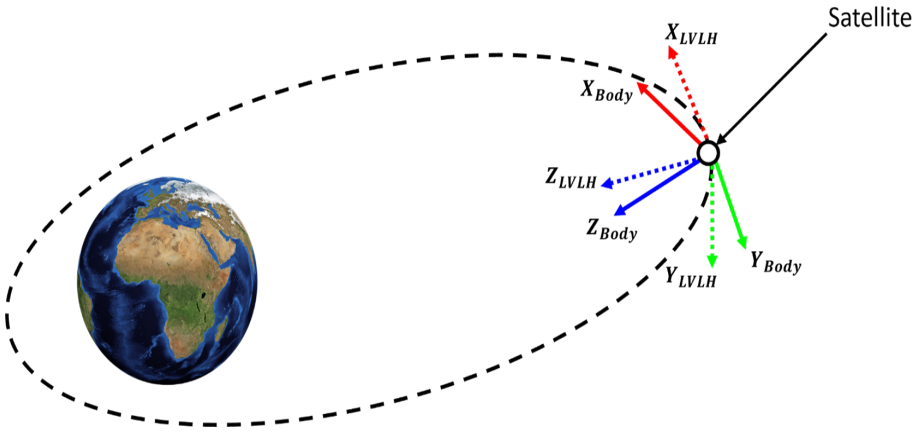


Figure 7: Coordinate system for attitude control mission

complete orbit, after which the mission concludes. The control command $\underline{\tau}$ for the reaction wheel system is derived following the methodology described in this article.[14] The motion of equation of satellite in orbit and torque command can be derived as below.

$$I\underline{\dot{\omega}} + \underline{\omega} \times I\underline{\omega} + A_s I_{RW} \underline{\dot{\Omega}}_{RW} + \underline{\omega} \times A_s I_{RW} \underline{\Omega}_{RW} = 0 \tag{1}$$

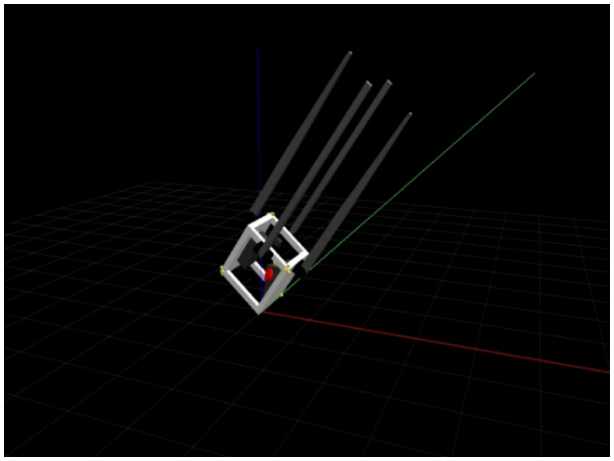


Figure 8: Graphic view of AstroROS for attitude control mission

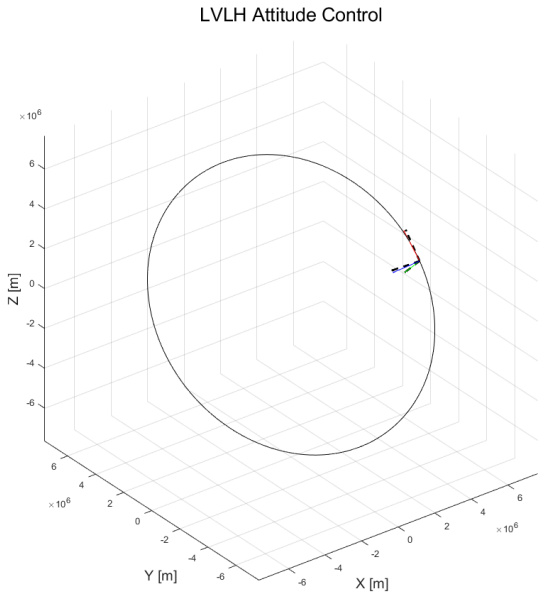


Figure 9: Overview of orbit propagation

$$\underline{\tau} = -(A_s I_{RW} \dot{\underline{\Omega}}_{RW} + \underline{\omega} \times A_s I_{RW} \underline{\Omega}_{RW}) \tag{2}$$

where A_s is the direction vector matrix of the pyramid configuration of 4 reaction wheels as follow:

$$A_s = \begin{bmatrix} \cos\beta & 0 & -\cos\beta & 0 \\ 0 & \cos\beta & 0 & -\cos\beta \\ \sin\beta & \sin\beta & \sin\beta & \sin\beta \end{bmatrix}, \beta = \frac{\pi}{4} \tag{3}$$

and I_{RW} is the inertia value of reaction wheel with respect to rotation axis, $\underline{\Omega}_{RW}$ is the velocity vector of 4 reaction wheels.

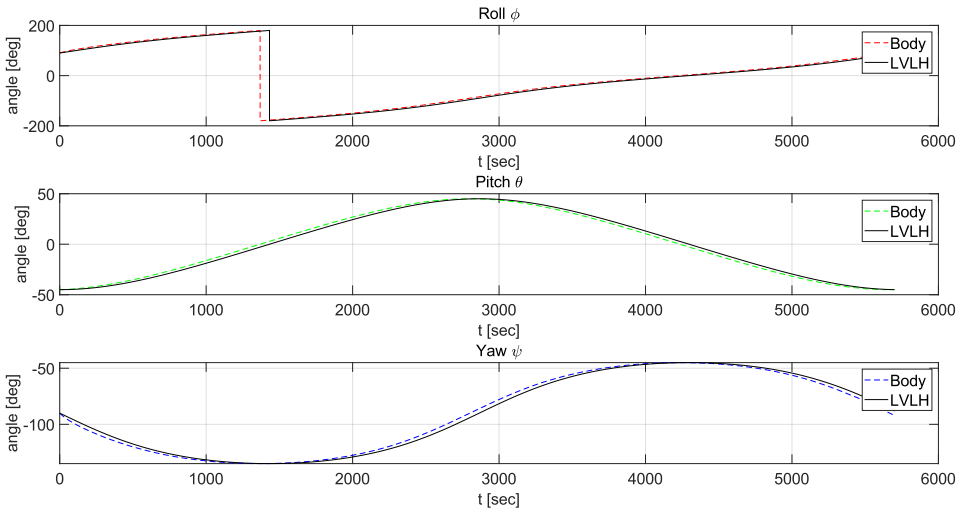


Figure 10: Attitude control result

The desired torque $\underline{\tau}$ for the mission, using a Proportional-Derivative (PD) controller, is defined as:

$$\underline{\tau} = K_p \underline{q}_e + K_d \underline{\omega}_e \tag{4}$$

where $\underline{q}_e$ and $\underline{\omega}_e$ represent the quaternion error and angular rate error between the body frame and the LVLH frame, respectively, and K_p and K_d are the PD gain constants. Figure 8 shows graphic rendering view of AstroROS and simulation results are shown from Figure 9 to Figure 10. Black lines in Figure 9 and Figure 10 represent trajectory or position and orientation of LVLH frame and colored lines correspond to the body frame. As shown in figure 10, although there are tracking errors, the results indicate that the satellite successfully executes the allocated mission.

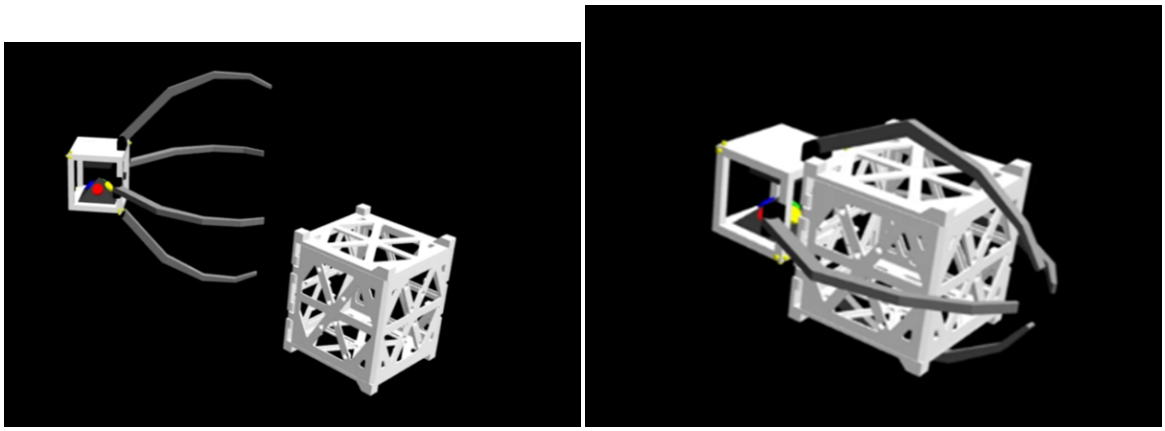
Case 2 : DEBRIS CAPTURE MISSION

The debris capture mission is designed to demonstrate the chaser satellite's capability to approach and capture a target object, simulating an Active Debris Removal(ADR) operation. Mission setup is shown in Figure 11-(a), that is the chaser rotates 45-degree on the z-axis and relative distance between two objects are 5 m.

The mission begins by aligning the chaser and the target along the x-axis of the chaser's body frame using the reaction wheel system. Once aligned, the chaser uses its thrusters to close the distance to the target, executing controlled maneuvers to ensure a safe approach.

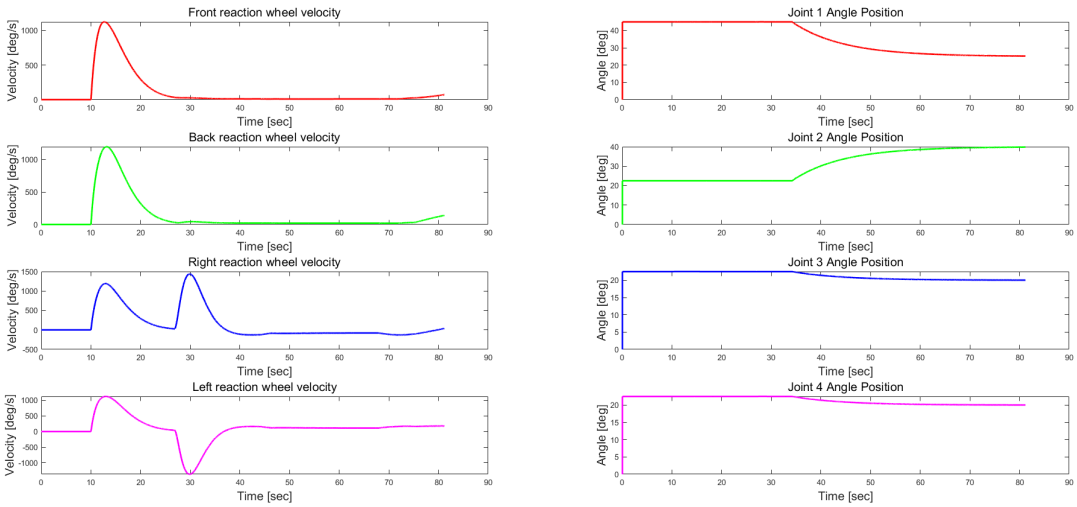
Upon reaching the vicinity of the target, the chaser deploys its tentacle mechanism to grasp the target securely. For attitude control, the reaction wheel system operates under a Proportional-Derivative (PD) control strategy, similar to the one used in attitude control mission. The thruster system employs a bang-bang control method, providing on-off thrust commands to adjust the chaser's trajectory and velocity during the approach phase.

The tentacle control is also handled through PD control, following a scheduled motion sequence for avoiding dangerous situation.



(a) Designed ADR mission in Physics Simulator (b) Chaser's capture motion for grasping target object

Figure 11: Case study for AstroROS validation with debris capture scenario



(a) Reaction wheel velocity log (b) Tentacle joint position log

Figure 12: Log data for reaction wheel velocity and tentacle joint angle position log

The tentacle is equipped with sensors and actuators that provide real-time feedback, allowing the control system to make necessary adjustments to ensure a secure grasp of the target object. As illustrated in Figure 11-(b), the chaser moves closer to the target, the combined use of the thruster and reaction wheel systems results successful attitude stabilization.

Figure 12 highlights the simulation results of reaction wheels and tentacle joint position. As shown, the reaction wheels adjust their rotational speeds dynamically in response to the PD control commands, ensuring precise attitude control throughout the mission. From this graph, we can find that alignment between chaser and target operates approximately from 10 seconds to 28 seconds and stabilize its attitude while the chaser approaches to the target using thrusters from 28 seconds to 40 seconds. As mentioned before,the tentacle's motion follows the pre-defined control sequence during the rest of time.

Overall, the demos show the capability of the AstroROS platform to model and validate critical mission scenarios involving dynamic interactions and control in space.

CONCLUSION

This article introduces the OOS simulation framework for the space robot system AstroROS.[15] AstroROS is capable of simulating various satellite missions by integrating ADCS, navigation system, and a space robot system with an orbit propagation model. Developed using ROS/Gazebo and MATLAB/Simulink, AstroROS has been validated through a virtual ADR mission involving attitude and position control, as well as the operation of a ClearSpace-like tentacle manipulator. The simulation results demonstrate that AstroROS can effectively simulate comprehensive space missions, as a valuable tool for researchers and engineers. For future work, AstroROS will be migrated from ROS1 to ROS2 so that it could be utilized in other frameworks like SpaceROS and the MATLAB/Simulink components will be revised as C++ for solely using ROS2. Additionally, AstroROS will be applied for Hardware-In-the-Loop Simulation (HILS) using Space Satellite Ground Testbed(SSGT) developed by USCL. The framework is accessible on the author's GitHub: https://github.com/MichaelBentleyOh/AstroROS.

ACKNOWLEDGMENTS

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Science and ICT.(No. NRF-2022M1A3B8073175).

REFERENCES

[1] S. T. K. C. Markus Wilde and M. Romano, "Kinematic and Dynamic Spacecraft Maneuver Simulators for Verification and Validation of Space Robotic Systems," *Pre print*, 2020.

[2] K. Hovell and S. Ulrich, "On Deep Reinforcement Learning for Spacecraft Guidance," *AIAA 2020 Scitech*, No. AIAA 2020-1600, 2020.

[3] R. JL, P. J., and F. L., "A ROS/Gazebo-based framework for simulation and control of on-orbit robotic systems," *73rd International Astronautical Congress (IAC-22)*, 2022, pp. 71–92.

[4] S. P. Kenneally, Patrick W. and H. Schaub., "Basilisk: A flexible, scalable and modular astrodynamics simulation framework.," *Journal of aerospace information systems*, 2020.

[5] K. Chow, N. Oune, J. Singh, D. Thomlinson, and T. A. Battista, "Simulating spacecraft docking and berthing using ROS and Gazebo," *AIAA SCITECH 2022*, 2022.

[6] N. A. Radford, P. Strawser, K. Hambuchen, J. S. Mehling, W. K. Verdeyen, A. S. Donnan, and J. Akinyode, "Valkyrie: Nasa's first bipedal humanoid robot," *Journal of Field Robotics*, Vol. 32, No. 3, 2015, pp. 397–419.

[7] A. Probe, A. Oyake, S. W. Chambers, M. Deans, G. Brat, N. B. Cramer, and K. Hambuchen, "Space ROS: An open-source framework for space robotics and flight software," *AIAA SCITECH 2023*, 2023.

[8] R. Biesbroek, S. Aziz, A. Wolahan, S.-f. Cipolla, M. Richard-Noca, and L. Piguet, *The clearspace-1 mission: ESA and clearspace team up to remove debris*. 2021.

[9] V. Carrara, "An open source satellite attitude and orbit simulator toolbox for Matlab," 2015, pp. 1–12.

[10] H. J. You, K. Kim, and D. Jung, "A Real-Time Simulator for Processor-In-the-Loop Simulation of Small Satellites," 2018.

[11] S. Ivaldi, V. Padois, and F. Nori, "Tools for dynamics simulation of robots: a survey based on user feedback," *arXiv preprint arXiv:1402.7050*, 2014.

[12] Y. T. T. Erez and E. Todorov, "Simulation tools for model-based robotics: Comparison of Bullet, Havok, MuJoCo, ODE and PhysX," *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 4397–4404.

[13] S. Chitta, E. Marder-Eppstein, W. Meeussen, V. Pradeep, A. R. Tsouroukdissian, J. Bohren, D. Coleman, B. Magyar, G. Raiola, M. Lüdtke, *et al.*, "ros_control: A generic and simple control framework for ROS," *Journal of Open Source Software*, Vol. 2, No. 20, 2017, pp. 456–456.

[14] D. Jung and P. Tsiotras, *A 3-dof experimental test-bed for integrated attitude dynamics and control research*. 2003.

[15] D. J. Minsik Oh, Heokjune You, "AstroROS," *https://github.com/MichaelBentleyOh/AstroROS*, 2024.